

1. Przygotowanie danych – Tabela płaska

Zaczynamy od stworzenia standardowej tabeli sprzedaz_flat i wypełnienia jej milionem testowych rekordów.

```
-- Tworzymy tabelę płaską
CREATE TABLE sprzedaz_flat (
  id SERIAL,
  data_sprzedazy DATE NOT NULL,
  kwota DECIMAL(10,2),
  klient_id INT
);
```

Wygeneruj poprzez dowolny model LLM załadowanie 1 mln losowych wierszy do tej tabeli

```
-- Dodajemy standardowy indeks na datę
```

2. Tworzenie architektury partycjonowanej

Zastosujemy **właściwe partycjonowanie** (według daty)

```
-- Tworzymy tabelę nadrzędną (logiczna struktura)
```

```
-- Tworzymy partycje fizyczne (np. kwartalne – q1, q2, q3, q4)
```

```
-- W PostgreSQL indeksy na tabeli nadrzędnej są automatycznie propagowane na partycje.
```

```
-- Przenosimy dane z tabeli płaskiej do partycjonowanej
```

3. Testy wydajności i "Partition Pruning"

Teraz sprawdzimy, jak zachowuje się planista zapytań w obu przypadkach przy użyciu EXPLAIN ANALYZE.

Scenariusz: Wyciągamy dane z konkretnego miesiąca (np. lutego).

```
-- Zapytanie do tabeli PŁASKIEJ
EXPLAIN ANALYZE
SELECT * FROM sprzedaz_flat
WHERE data_sprzedazy BETWEEN '2025-02-01' AND '2025-02-28';

-- Czy baza przeszukuje indeks na całej tabeli nadrzędnej?
```

```
-- Zapytanie do tabeli PARTYCJONOWANEJ
```

```
EXPLAIN ANALYZE
```

```
SELECT * FROM sprzedaz_part
```

```
WHERE data_sprzedazy BETWEEN '2025-02-01' AND '2025-02-28';
```

```
-- Zobacz, czy baza uzyskała dostęp TYLKO do partycji 'sprzedaz_q1'.
```

```
-- Czy pozostałe partycje (q2, q3, q4) zostały całkowicie zignorowane (Pruning).
```

Zadanie :

Spróbuj zmienić datę sprzedaży jednego z wierszy(poprzez update) w taki sposób, aby "przeskoczył" do innej partycji. PostgreSQL automatycznie wykona operację DELETE + INSERT między partycjami.